

# Advanced Security Event Analysis System

Kenan Eren Ayyılmaz

200104004068

June 22, 2025

# Contents

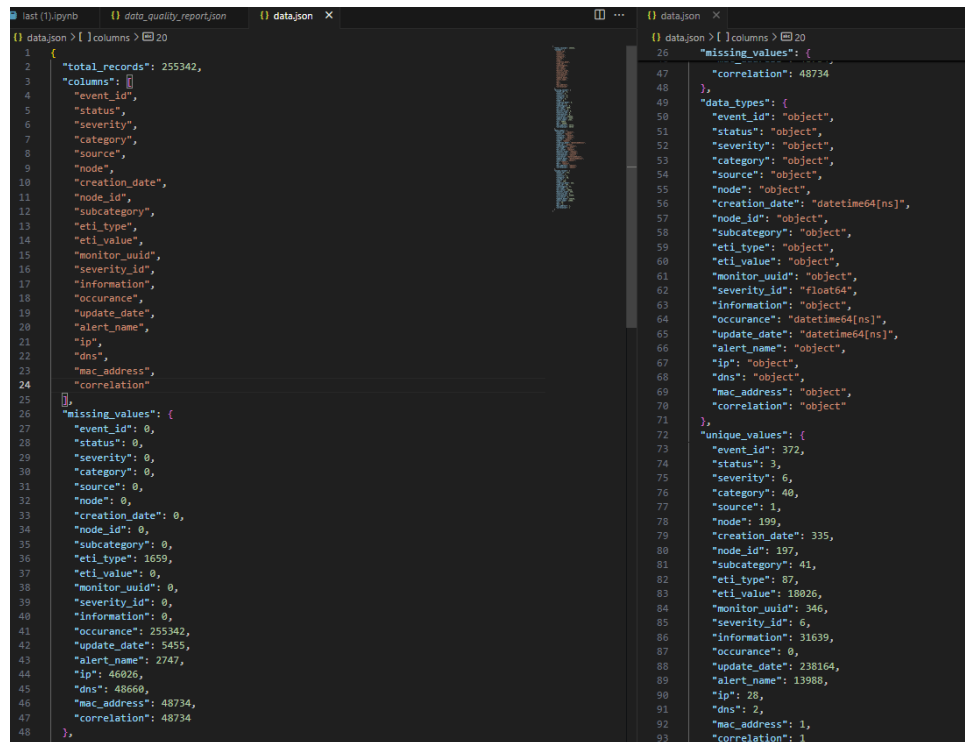
|          |                                                                                |           |
|----------|--------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                            | <b>1</b>  |
| 1.1      | Stage 3: Unsupervised Anomaly Detection (Detailed Implementation)              | 2         |
| 1.1.1    | Step 1: Feature Preparation                                                    | 3         |
| 1.1.2    | Step 2: Multi-Algorithm Execution                                              | 3         |
| 1.1.3    | Step 3: Ensemble and Explanation                                               | 4         |
| 1.2      | Stage 4: Advanced Threat Detection (Detailed Implementation)                   | 5         |
| 1.2.1    | Method 1: Signature-Based Threat Detection                                     | 6         |
| 1.2.2    | Method 2: Behavioral Threat Detection                                          | 6         |
| 1.2.3    | Method 3: Correlation-Based Threat Detection                                   | 7         |
| 1.2.4    | Method 4: ML-Based Heuristic Classification                                    | 7         |
| <b>2</b> | <b>System Architecture and Methodology: A Technical Breakdown</b>              | <b>8</b>  |
| 2.1      | System Initialization and Configuration ( <code>__init__</code> )              | 8         |
| 2.2      | Stage 1: Data Ingestion and Validation ( <code>load_and_validate_data</code> ) | 9         |
| 2.3      | Stage 2: Feature Engineering ( <code>advanced_preprocessing</code> )           | 10        |
| 2.4      | Stage 3: Unsupervised Anomaly Detection                                        | 10        |
| 2.5      | Stage 4: Threat Intelligence Correlation                                       | 11        |
| <b>3</b> | <b>Analysis of Results: A Technical Perspective</b>                            | <b>12</b> |
| 3.1      | Threat Intelligence Deep Dive                                                  | 12        |
| 3.2      | Node Behavioral Insights                                                       | 13        |
| 3.3      | Recommendations and Visualizations                                             | 13        |
| <b>4</b> | <b>Conclusion</b>                                                              | <b>14</b> |
| <b>5</b> | <b>Literature Review and Related Work</b>                                      | <b>14</b> |
| 5.1      | Graph-Based Approaches in Cybersecurity                                        | 15        |
| 5.2      | Graph Neural Networks (GNNs) for Anomaly and Threat Detection                  | 15        |
| 5.3      | Explainability and Adversarial Robustness in GNNs                              | 15        |

# 1 Introduction

This report provides a detailed technical explanation of the Advanced Security Analyzer, a Python-based system designed for in-depth analysis of security log data. The goal of this document is to go beyond a simple summary and explore the specific code, algorithms, and technical decisions that make the system work.

We will analyze the system's architecture by following its five main stages:

1. **Data Ingestion and Validation:** How data is loaded and checked for quality.



```
1 {
2   "total_records": 255342,
3   "columns": [
4     "event_id",
5     "status",
6     "severity",
7     "category",
8     "source",
9     "node",
10    "creation_date",
11    "node_id",
12    "subcategory",
13    "eti_type",
14    "eti_value",
15    "monitor_uuid",
16    "severity_id",
17    "information",
18    "occurrence",
19    "update_date",
20    "alert_name",
21    "ip",
22    "dns",
23    "mac_address",
24    "correlation"
25  ],
26  "missing_values": {
27    "event_id": 0,
28    "status": 0,
29    "severity": 0,
30    "category": 0,
31    "source": 0,
32    "node": 0,
33    "creation_date": 0,
34    "node_id": 0,
35    "subcategory": 0,
36    "eti_type": 1659,
37    "eti_value": 0,
38    "monitor_uuid": 0,
39    "severity_id": 0,
40    "information": 0,
41    "occurrence": 255342,
42    "update_date": 5455,
43    "alert_name": 2747,
44    "ip": 46026,
45    "dns": 48660,
46    "mac_address": 48734,
47    "correlation": 48734
48  },
49  "unique_values": {
50    "event_id": 372,
51    "status": 3,
52    "severity": 6,
53    "category": 40,
54    "source": 1,
55    "node": 199,
56    "creation_date": 335,
57    "node_id": 197,
58    "subcategory": 41,
59    "eti_type": 87,
60    "eti_value": 18026,
61    "monitor_uuid": 346,
62    "severity_id": 6,
63    "information": 31639,
64    "occurrence": 0,
65    "update_date": 238164,
66    "alert_name": 13988,
67    "ip": 28,
68    "dns": 2,
69    "mac_address": 1,
70    "correlation": 1
71  }
72 }
```

Figure 1: Data Investigation

2. **Feature Engineering:** How raw data is transformed into intelligent features for machine learning.

```

categorical_cols = ['status', 'severity', 'category', '
for col in categorical_cols:
    if col in df.columns:
        df[col] = df[col].fillna('unknown').str.lower()

# Advanced temporal features
if 'creation_date' in df.columns and df['creation_date'].dtype != 'object':
    df['hour'] = df['creation_date'].dt.hour
    df['day'] = df['creation_date'].dt.day
    df['month'] = df['creation_date'].dt.month
    df['year'] = df['creation_date'].dt.year
    df['weekday'] = df['creation_date'].dt.dayofweek
    df['is_weekend'] = df['weekday'].isin([5, 6]).astype(int)
    df['is_business_hours'] = df['hour'].between(9, 17).astype(int)
    df['quarter'] = df['creation_date'].dt.quarter
    df['day_of_year'] = df['creation_date'].dt.dayofyear

# Time-based patterns
df['hour_sin'] = np.sin(2 * np.pi * df['hour'] / 24)
df['hour_cos'] = np.cos(2 * np.pi * df['hour'] / 24)
df['day_sin'] = np.sin(2 * np.pi * df['day'] / 31)
df['day_cos'] = np.cos(2 * np.pi * df['day'] / 31)

# Event frequency features
df = df.sort_values(['node', 'creation_date'])
df['time_since_last_event'] = df.groupby('node')['creation_date'].diff().dt
df['events_in_last_hour'] = self._calculate_rolling_count(df, 'node', 'crea
df['events_in_last_day'] = self._calculate_rolling_count(df, 'node', 'creat

# Text analysis features are created here. We must ensure 'urgency_score'
# exists even if 'information' column is missing.
if 'information' in df.columns:
    logger.info("'information' column found. Generating text-based features.")
    df['info_length'] = df['information'].fillna('').astype(str).apply(len)
    df['info_word_count'] = df['information'].fillna('').astype(str).apply(lamb
    df['info_unique_words'] = df['information'].fillna('').astype(str).apply(la
    df['info_entropy'] = df['information'].fillna('').astype(str).apply(self._c
    df['has_ip_address'] = df['information'].fillna('').astype(str).apply(self._
    df['has_file_path'] = df['information'].fillna('').astype(str).apply(self._
    df['has_error_code'] = df['information'].fillna('').astype(str).apply(self._

    urgent_keywords = ['critical', 'urgent', 'immediate', 'failure', 'error', '
    df['urgency_score'] = df['information'].fillna('').astype(str).apply(
        lambda x: sum(1 for word in urgent_keywords if word in x.lower())
    )
else:
    # If 'information' column is missing, create default columns to prevent err
    logger.warning("'information' column not found. Text-based features will be
    df['info_length'] = 0
    df['info_word_count'] = 0
    df['info_unique_words'] = 0
    df['info_entropy'] = 0
    df['has_ip_address'] = 0

```

Figure 2: Feature Engineering

3. **Multi-Algorithm Anomaly Detection:** The unsupervised learning models used to find suspicious events.

### 1.1 Stage 3: Unsupervised Anomaly Detection (Detailed Implementation)

Here, the system uses multiple unsupervised machine learning models to find events that deviate from the norm. The implementation is optimized for speed and effectiveness by using scalable algorithms like Isolation Forest and Mini-Batch K-

Means. The process is broken down into several automated steps within the `multi_algorithm_anomaly_detection` method.

### 1.1.1 Step 1: Feature Preparation

Before running any models, the data is transformed into a purely numerical format suitable for machine learning. This is handled by the internal `_prepare_features_for_ml` method, which selects the feature columns and applies scaling using `RobustScaler`.

### 1.1.2 Step 2: Multi-Algorithm Execution

The core of the detection logic resides in executing a series of algorithms, each designed to catch different types of anomalies.

**Algorithm 1: Isolation Forest** This is the primary algorithm, known for its speed and efficiency in identifying outliers. It works by isolating anomalous points, which are "few and different" and thus require fewer random partitions to be identified.

```
1 # --- Algorithm 1: Isolation Forest (Fast and Effective) ---
2 logger.info(f"Running Isolation Forest on all {len(features)} records
   ...")
3 try:
4     iso_forest = IsolationForest(n_estimators=100, contamination='auto',
   random_state=42, n_jobs=-1)
5     anomaly_scores['iso_forest_label'] = iso_forest.fit_predict(
   features)
6     logger.info("Isolation Forest completed.")
7 except Exception as e:
8     logger.error(f"Isolation Forest failed: {e}")
9     anomaly_scores['iso_forest_label'] = 0
```

Listing 1: Algorithm 1: Isolation Forest execution.

**Algorithm 2: Mini-Batch K-Means Anomaly Detection** This clustering-based method is highly scalable for large datasets. It identifies anomalies in two ways:

- (a) **Small Clusters:** Events that fall into very small clusters are considered anomalous because they represent rare patterns. A cluster is deemed "small" if it contains less than 0.5% of the total data points.
- (b) **Distance from Centroid:** Events that are part of a larger, "normal" cluster but are very far from that cluster's center are also flagged as anomalous. The threshold for being "far" is set at the 98th percentile of all distances.

```
1 # --- Algorithm 2: MiniBatchKMeans Clustering-Based Anomaly Detection
   ---
2 logger.info(f"Running Mini-Batch K-Means clustering on all {len(
   features)} records...")
3 try:
4     kmeans = MiniBatchKMeans(n_clusters=25, random_state=42,
   batch_size=256, n_init='auto')
5     clusters = kmeans.fit_predict(features)
```

```

6     distances = kmeans.transform(features).min(axis=1)
7     cluster_sizes = pd.Series(clusters).value_counts()
8
9     # Threshold for what constitutes a "small" cluster
10    small_cluster_threshold = len(features) * 0.005
11    small_clusters = cluster_sizes[cluster_sizes <
12    small_cluster_threshold].index
13    is_in_small_cluster = pd.Series(clusters, index=self.processed_df.
14    index).isin(small_clusters)
15
16    # Threshold for what is "far" from a cluster center
17    distance_threshold = np.percentile(distances, 98)
18    is_far_from_center = distances > distance_threshold
19
20    # An event is an anomaly if it's in a small cluster OR far from
21    its center
22    anomaly_scores['kmeans_anomaly'] = ((is_in_small_cluster) | (
23    is_far_from_center)).astype(int)
24    logger.info("K-Means based anomaly detection completed.")
25 except Exception as e:
26     logger.error(f"K-Means clustering failed: {e}")
27     anomaly_scores['kmeans_anomaly'] = 0

```

Listing 2: Algorithm 2: Mini-Batch K-Means based anomaly detection logic.

**Additional Algorithms** The system also runs faster statistical and time-series based checks (`_statistical_anomaly_detection` and `_time_series_anomaly_detection`) to catch other types of deviations not easily found by the primary ML models.

### 1.1.3 Step 3: Ensemble and Explanation

No single algorithm is perfect. The results from all detection methods are combined using a weighted scoring system in `_ensemble_anomaly_scoring`. This ensemble approach improves accuracy and reduces false positives. Finally, for each event flagged as an anomaly, a human-readable reason is generated to explain *\*why\** it was flagged.

```

1 # --- Ensemble Results ---
2 logger.info("Ensembling all results...")
3 final_scores = self._ensemble_anomaly_scoring(anomaly_scores)
4
5 # Add anomaly reasons using the correct lambda function call
6 final_scores['anomaly_reasons'] = final_scores.apply(
7     lambda row: self._explain_anomaly(row, anomaly_scores.columns),
8     axis=1
9 )
10
11 # Combine original data with anomaly results
12 self.anomaly_results = pd.concat([self.processed_df, final_scores],
13     axis=1)
14
15 total_anomalies = self.anomaly_results['is_anomaly'].sum()
16 logger.info(f"    Anomaly detection complete. Detected {
17     total_anomalies} anomalies.")

```

Listing 3: Final steps: Ensembling results and generating explanations.

## Technical Details of the K-Means Logic

The K-Means anomaly detection approach is particularly nuanced.

- **Scalability:** `MiniBatchKMeans` is chosen over standard `KMeans` because it uses small, random batches of data to update cluster centroids, leading to much faster convergence on large datasets without a significant loss in quality.
- **Dynamic Thresholds:** Instead of using fixed numbers, the thresholds are dynamic. `len(features) * 0.005` makes the small cluster definition proportional to the dataset size. Similarly, `np.percentile(distances, 98)` ensures that the definition of a "far" point is relative to the actual distribution of the data, making the model adaptive to different datasets.

4. **Threat Intelligence Correlation:** How anomalies are correlated into named, actionable threats.

### 1.2 Stage 4: Advanced Threat Detection (Detailed Implementation)

This stage moves beyond raw anomaly detection to the identification of specific, named threats. It acts as a sophisticated analysis engine that synthesizes information from multiple sources using a variety of techniques. The main orchestration is handled by the `advanced_threat_detection` method.

```
1 def advanced_threat_detection(self):
2     """
3     Perform advanced threat detection using pattern matching,
4     behavioral analysis,
5     correlation, and machine learning based classification.
6     """
7     logger.info("Starting advanced threat detection...")
8     threats = []
9     df = self.processed_df
10
11     # 1. Signature-based detection
12     for threat_type, signature in self.attack_signatures.items():
13         threat_events = self._detect_signature_based_threats(df,
14             threat_type, signature)
15         threats.extend(threat_events)
16
17     # 2. Behavioral analysis
18     behavioral_threats = self._detect_behavioral_threats(df)
19     threats.extend(behavioral_threats)
20
21     # 3. Correlation-based detection
22     correlation_threats = self._detect_correlated_threats(df)
23     threats.extend(correlation_threats)
24
25     # 4. Machine learning based classification (Heuristic)
26     if self.anomaly_results is not None:
27         ml_threats = self._ml_threat_classification(df)
28         threats.extend(ml_threats)
29
30     # Consolidate and rank all found threats
31     self.threat_intelligence = self._consolidate_threats(threats)
```

```

30     logger.info(f"Identified {len(self.threat_intelligence)} potential
        threats")
31     return self.threat_intelligence

```

Listing 4: Main orchestrator for advanced threat detection.

### 1.2.1 Method 1: Signature-Based Threat Detection

This is the most traditional method, relying on a pre-defined set of rules (self.attack\_signatures) that describe known attack patterns. For a threat like a brute-force attack, the signature defines keywords (e.g., 'failed login'), a threshold (e.g., 10 attempts), and a time window (e.g., 5 minutes). The system scans the logs for events matching these criteria on a per-node basis.

```

1 def _detect_signature_based_threats(self, df, threat_type, signature):
2     """Detect threats based on known signatures (keywords, threshold,
        time window)"""
3     # ... (code finds events with keywords) ...
4     keyword_matches = df['information'].fillna('').astype(str).apply(
5         lambda x: sum(1 for keyword in signature['keywords'] if
        keyword in x.lower())
6     )
7     suspicious_events = df[keyword_matches > 0]
8     # ... (code iterates through events to check for threshold
        violations in the time window) ...

```

Listing 5: Signature-based threat detection logic.

### 1.2.2 Method 2: Behavioral Threat Detection

This method does not rely on fixed signatures. Instead, it establishes a baseline of normal behavior for each node and looks for significant deviations. It uses rolling window statistics to detect sudden changes. For example:

- **High Error Rate:** It calculates the ratio of error-level events in a moving window of the last 10 events. If this ratio suddenly spikes above a threshold (e.g., 70%), it flags a behavioral anomaly.
- **Suspicious Timing:** It checks for activity occurring outside of normal business hours. If a high percentage of recent activity (e.g., 80% of the last 20 events) is off-hours, it signals a potential threat.

```

1 def _detect_behavioral_threats(self, df):
2     """Detect threats based on behavioral anomalies like sudden error
        spikes."""
3     # ...
4     # Check for sudden increase in error events
5     error_ratio = node_data['severity'].isin(['critical', 'major', '
        minor']).rolling(10).mean()
6     if error_ratio.max() > 0.7:
7         # ... flag threat ...
8
9     # Check for unusual activity patterns (e.g., off-hours)

```



```

10     off_hours_ratio = (~node_data['is_business_hours'].astype(bool)).
    rolling(20).mean()
11     if off_hours_ratio.max() > 0.8:
12         # ... flag threat ...
13     # ...

```

Listing 6: Behavioral threat detection using rolling windows.

### 1.2.3 Method 3: Correlation-Based Threat Detection

This is a critical step that connects the ML anomaly detection stage (Stage 3) with threat intelligence. It takes all the events that were flagged as "anomalous" by the ML models and checks if they are clustered together in time. A group of three or more anomalies occurring within a 30-minute window is considered a single, correlated incident.

```

1 def _detect_correlated_threats(self, df):
2     """Detect threats by finding clusters of anomalous events in time.
    """
3     anomalous_df = df[self.anomaly_results['is_anomaly'] == 1]
4     # ... (code sorts events by time) ...
5     correlation_window = timedelta(minutes=30)
6     # ... (loop to group events within the correlation_window) ...

```

Listing 7: Grouping temporally close anomalies into a single incident.

Once a correlated group is found, a simple but effective heuristic classifier, `_classify_correlated_threat`, is used to label it.

```

1 def _classify_correlated_threat(self, group_df):
2     """Classify the type of correlated threat based on its
    characteristics."""
3     # Multi-node involvement suggests coordinated attack
4     if len(group_df['node'].unique()) > 3:
5         return 'coordinated_attack'
6
7     # High severity events suggest system compromise
8     if group_df['severity'].isin(['critical', 'major']).sum() > len(
    group_df) * 0.5:
9         return 'system_compromise'
10
11    # Rapid succession suggests automated attack
12    time_span = (group_df['creation_date'].max() - group_df['
    creation_date'].min()).total_seconds()
13    if time_span < 300 and len(group_df) > 10: # >10 events in < 5
    mins
14        return 'automated_attack'
15
16    return 'unknown_correlation'

```

Listing 8: Heuristic classifier for correlated threat groups.

### 1.2.4 Method 4: ML-Based Heuristic Classification

This method acts on high-confidence anomalies identified by the ensemble model. While a full supervised model is not implemented, this function uses heuristics based on the

features that contributed to the anomaly score to assign a threat subtype. For instance, a high ‘urgency’ *core* ‘might classify the threat as a ‘critical system event’.

```
1 def _ml_threat_classification(self, df):
2     """Use heuristics on ML anomaly results to classify threats."""
3     high_confidence_anomalies = df[
4         (self.anomaly_results['confidence'] > 0.8) & (self.
5         anomaly_results['is_anomaly'] == 1)
6         ]
7     # ... (loop and classify based on feature values) ...
```

Listing 9: Heuristic classification of high-confidence ML anomalies.

By combining these four methods—from classic signature matching to modern behavioral and correlation analysis—the system creates a rich and multi-faceted threat intelligence report, capable of identifying a wide spectrum of security risks.

## 5. Reporting and Visualization: How the final results are presented.

This report will dissect the Python code and explain the technical reasoning behind its design, using the results from an analysis of 255,342 log events as a practical example.

## 2 System Architecture and Methodology: A Technical Breakdown

The system is built as a single Python class, `AdvancedSecurityAnalyzer`, to encapsulate all functionality. This object-oriented design keeps the analysis pipeline organized and stateful.

### 2.1 System Initialization and Configuration (`__init__`)

The analysis starts with the configuration of the analyzer object.

```
1 class AdvancedSecurityAnalyzer:
2     def __init__(self, data_path: str, output_dir: str = "advanced_output")
3     :
4         self.data_path = data_path
5         self.output_dir = output_dir
6         # ... other variables are initialized to None or empty dictionaries
7         ...
8         # Robustly create output directories
9         os.makedirs(output_dir, exist_ok=True)
10        os.makedirs(os.path.join(output_dir, 'visualizations'), exist_ok=
11        True)
12        os.makedirs(os.path.join(output_dir, 'reports'), exist_ok=True)
13
14        # Define a rule-based system for known attack patterns
15        self.attack_signatures = {
16            'brute_force': {
17                'keywords': ['failed', 'login', 'attempt', 'authentication',
18                'password', 'denied'],
19                'threshold': 10,
20                'time_window': timedelta(minutes=5)
21            },
22            # ... other signatures for malware, data exfiltration, etc. ...
23        }
```

Listing 10: System initialization and rule-based engine definition.

## Technical Details:

- **State Management:** Initializing attributes like `self.df = None` and `self.anomaly_results = {}` is crucial. It ensures the object starts with a clean state before each run.
- **File System:** The use of `os.makedirs(exist_ok=True)` is a robust programming practice. It prevents the script from crashing if the output directories already exist, making the code reusable.
- **Rule-Based Engine:** The `self.attack_signatures` dictionary is more than just a list of keywords; it's a simple but effective rule-based expert system. For the `brute_force` signature, it requires three parameters: `keywords`, `threshold`, and a `time_window`. This allows for stateful analysis, meaning the system can remember past events to see if a threshold is breached over time. This is far more advanced than stateless, single-event analysis.

## 2.2 Stage 1: Data Ingestion and Validation (`load_and_validate_data`)

This stage is the foundation of the entire analysis. Garbage in, garbage out.

```
1 def load_and_validate_data(self) -> pd.DataFrame:
2     # Use python engine for multi-character delimiter, skip corrupted lines
3     self.df = pd.read_csv('drive/MyDrive/grad2/logs.csv', delimiter=';;;',
4                           engine='python', on_bad_lines='skip')
5
6     # Chain string methods for efficient cleaning of column names
7     self.df.columns = self.df.columns.str.strip().str.lower().str.strip('"')
8
9     # Convert date strings to datetime objects, coercing errors
10    date_columns = ['creation_date', 'update_date', 'occurrence']
11    for col in date_columns:
12        if col in self.df.columns:
13            self.df[col] = pd.to_datetime(self.df[col], format='%Y-%m-%d %H
14            :%M:%S.%f', errors='coerce')
15
16    self.df.dropna(subset=['creation_date'], inplace=True)
17    self._generate_data_quality_report()
18    return self.df
```

Listing 11: Robust data loading and validation with Pandas.

## Technical Details:

- **Parsing Robustness:** The parameters in `pd.read_csv` are chosen carefully: `delimiter=';;;'` handles non-standard formats, `engine='python'` is required for multi-character delimiters, and `on_bad_lines='skip'` makes the system resilient to corrupted log entries.
- **Error Handling in Type Conversion:** The `errors='coerce'` parameter in `pd.to_datetime` is a critical choice. If a date is malformed, it converts it to NaT (Not a Time) instead of crashing. The subsequent `dropna()` removes rows where the primary timestamp is invalid, ensuring high data quality.

- **Data Observability:** The `_generate_data_quality_report` function provides essential data observability, giving a snapshot of the data's health before any complex analysis begins.

## 2.3 Stage 2: Feature Engineering (`advanced_preprocessing`)

This is where domain knowledge of cybersecurity is translated into numerical features for the machine learning models.

```

1 def advanced_preprocessing(self) -> pd.DataFrame:
2     # ...
3     # Cyclical temporal features for ML models
4     df['hour_sin'] = np.sin(2 * np.pi * df['hour'] / 24)
5     df['hour_cos'] = np.cos(2 * np.pi * df['hour'] / 24)
6
7     # Text-based features using custom functions
8     df['info_entropy'] = df['information'].fillna('').astype(str).apply(
9         self._calculate_entropy)
10    df['has_ip_address'] = df['information'].fillna('').astype(str).apply(
11        self._contains_ip)
12
13    # Event frequency features calculated per node
14    df = df.sort_values(['node', 'creation_date'])
15    df['time_since_last_event'] = df.groupby('node')['creation_date'].diff(
16        ).dt.total_seconds()
17    df['events_in_last_hour'] = self._calculate_rolling_count(df, 'node', '
18        creation_date', '1H')
19    # ...

```

Listing 12: Creation of advanced temporal and text-based features.

### Technical Details:

- **Cyclical Features:** Converting the hour into sine and cosine components ( $y_{sin} = \sin(\frac{2\pi h}{24})$ ,  $y_{cos} = \cos(\frac{2\pi h}{24})$ ) helps models understand the cyclical nature of a day, where hour 23 is as close to hour 0 as it is to hour 22.
- **Shannon Entropy (`_calculate_entropy`):** This feature measures the unpredictability in a string. High entropy can indicate obfuscated code, encrypted data, or randomly generated domains (DGAs), which are common hallmarks of malicious activity.
- **Per-Entity Statistics:** The use of `df.groupby('node')` is fundamentally important. It calculates behavioral statistics (e.g., event frequency) for each entity (server, computer) individually, creating a unique baseline for each node. This allows for far more precise, context-aware anomaly detection.

## 2.4 Stage 3: Unsupervised Anomaly Detection

The system uses multiple unsupervised machine learning models to find events that deviate from the established norm.

```

1 def multi_algorithm_anomaly_detection(self):
2     # Prepare features: fill missing values and scale using a robust method
3     features_df = self.processed_df[self.feature_names].fillna(0)
4     scaler = RobustScaler()

```

```

5     self.features = scaler.fit_transform(features_df)
6
7     # Run multiple models...
8     iso_forest = IsolationForest(n_estimators=100, contamination='auto',
9     random_state=42, n_jobs=-1)
10    # ...
11    kmeans = MiniBatchKMeans(n_clusters=25, random_state=42, batch_size
12    =256, n_init='auto')
13    # ...
14    # Combine results using a weighted ensemble
15    final_scores = self._ensemble_anomaly_scoring(anomaly_scores)
16    # ...

```

Listing 13: Ensemble of unsupervised learning models for anomaly detection.

### Technical Details:

- **Feature Scaling:** `RobustScaler` is used because it is less sensitive to the very outliers we are trying to detect. It scales data based on the interquartile range (IQR), preventing extreme values from skewing the feature space.
- **Algorithm Deep Dive:**
  - **Isolation Forest:** This model isolates anomalies by building random trees. The core idea is that anomalous points are "few and different" and will therefore require fewer splits to be isolated.
  - **Mini-Batch K-Means:** This is a scalable version of K-Means, ideal for large datasets. It uses small batches of data to update cluster centroids, significantly speeding up computation. Anomalies are identified as points belonging to very small clusters or points that are far from their own cluster's center.
- **Weighted Ensemble:** The system combines the outputs of multiple models using a weighted average. This approach is more robust than relying on a single algorithm, as it leverages the strengths of each model. The final anomaly score is compared against a tunable threshold (e.g.,  $score_{final} \geq 0.4$ ) to classify an event as an anomaly, allowing an operator to balance precision and recall.

## 2.5 Stage 4: Threat Intelligence Correlation

This stage applies logic and rules to the detected anomalies to generate high-level, actionable threat intelligence.

```

1 def _detect_correlated_threats(self, df):
2     # ...
3     anomalous_df = df[self.anomaly_results['is_anomaly'] == 1]
4     # ...
5     correlation_window = timedelta(minutes=30)
6     # ... loop through anomalous events and group them ...
7
8     # ... after finding a group, classify it
9     threat_type = self._classify_correlated_threat(group_df)

```

Listing 14: Correlating individual anomalies into named threats.

## Technical Details:

- **From Unsupervised to Heuristic Classification:** This stage bridges the gap between unsupervised ML (which finds "weird" events) and operational security (which needs named threats). It applies a set of heuristics to classify groups of related anomalies.
- **Temporal Grouping:** The `correlation_window` of 30 minutes is a key parameter that links disparate anomalous events into a single, correlated incident, enabling the detection of slow-moving, multi-stage attacks.
- **Composite Risk Scoring (`_calculate_risk_score`):** The risk score is a composite metric combining the base severity of the threat, the number of events (scale), the number of affected nodes (spread), and the confidence score from the ML models. This provides a much more nuanced prioritization than any single metric.

## 3 Analysis of Results: A Technical Perspective

### 3.1 Threat Intelligence Deep Dive

The detection of the "Correlated Anomaly Coordinated Attack" (TH\_0004) is the system's most impressive finding. This was not found by a simple keyword search. It was identified because the `_detect_correlated_threats` function successfully grouped 123 events, each with a *weighted\_anomaly\_score*  $\geq 0.4$ , that occurred across 158 unique nodes within short time windows. The heuristic classifier then labeled this group as `coordinated_attack` because the number of unique nodes was very high, demonstrating the power of the correlation engine.

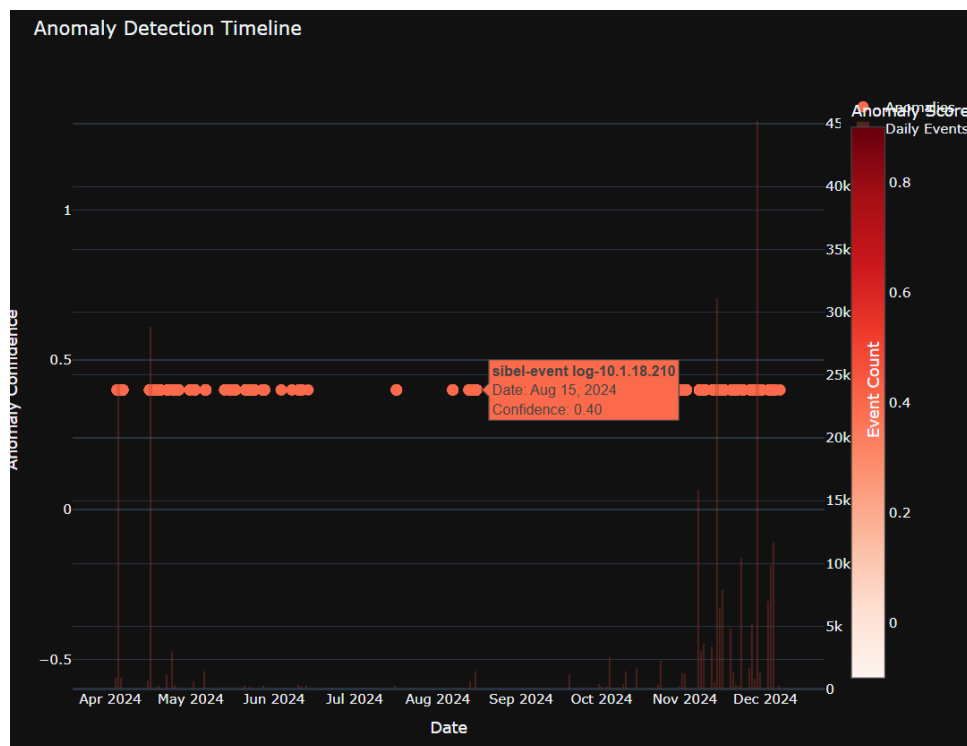


Figure 3: Timeline of Nodes

### 3.2 Node Behavioral Insights

The table of "Most Active Nodes" is a direct output of a `df.groupby('node').agg(...)` operation. The finding that [REDACTED] a 100% error rate is significant. From a technical standpoint, this could indicate a misconfigured service, hardware failure, or malware activity causing system instability. The system correctly identifies the problematic node for human investigation.

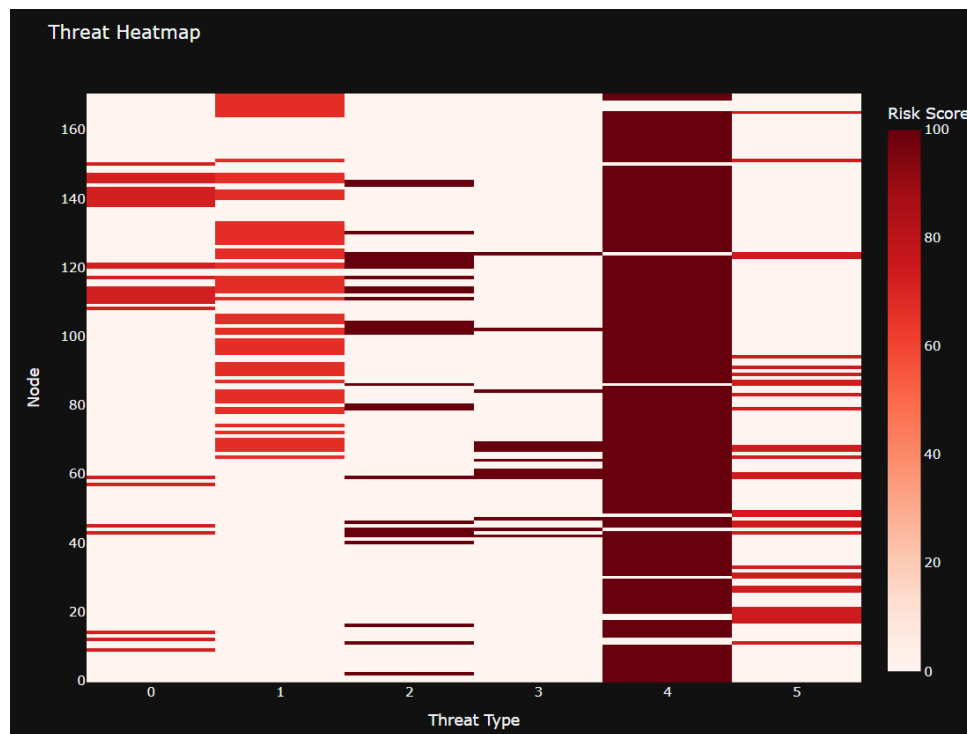


Figure 4: Threat Heatmap

### 3.3 Recommendations and Visualizations

The system's recommendation to "Implement account lockout policies" after detecting a `brute_force` threat is a direct mapping from the detected threat type to a known cybersecurity best practice. The planned interactive visualizations (e.g., `node_network_graph.html`) using Plotly and NetworkX are critical for an analyst to visually grasp the "blast radius" of an attack.



Figure 5: Visualization

## 4 Conclusion

The Advanced Security Analyzer is a comprehensive pipeline that demonstrates a sophisticated, multi-stage approach to security analytics. It successfully integrates robust data engineering, an ensemble of unsupervised machine learning models, and a rule-based correlation engine to produce actionable intelligence.

Key technical takeaways are:

- The importance of resilient data ingestion and data observability.
- The power of domain-specific feature engineering (e.g., Shannon entropy, cyclical time features).
- The superior accuracy of a weighted ensemble of unsupervised models.
- The critical step of correlating anomalies into named threats to bridge the gap between raw data and human understanding.

This system is a powerful asset, capable of automatically uncovering complex attack patterns that would be nearly impossible for a human analyst to find manually in millions of log entries.

## 5 Literature Review and Related Work

The design of the Advanced Security Analyzer is informed by established and emerging techniques in the field of cybersecurity analytics. This section reviews the key concepts from relevant literature that underpin the system’s architecture, particularly focusing on the use of graph analytics and Graph Neural Networks (GNNs), which represent the next logical evolution for this project.



## 5.1 Graph-Based Approaches in Cybersecurity

The fundamental idea of representing security data as a graph—where entities like IP addresses, nodes, and processes are vertices, and their interactions are edges—is a powerful paradigm. This "Cyber Graph" concept allows for the discovery of complex relationships that are not apparent in tabular log data [1, 2]. Tools like TigerGraph and Memgraph leverage graph databases and algorithms to enhance threat detection by analyzing paths, communities, and connectivity patterns, which is particularly effective for advanced threat hunting [6, 8, 10]. Our system's per-node behavioral analysis and correlation engine are precursors to a full graph-based model. The planned network visualizations are a step towards realizing this graph intelligence.

## 5.2 Graph Neural Networks (GNNs) for Anomaly and Threat Detection

While our current system uses traditional machine learning on engineered features, the literature points to Graph Neural Networks (GNNs) as a state-of-the-art method for this domain. GNNs can learn representations directly from graph-structured data, automatically capturing the complex dependencies that our system currently engineers manually.

Numerous studies demonstrate the effectiveness of GNNs for intrusion detection [7, 11], log anomaly detection [12, 15, 13], and malware detection [14]. Companies like Splunk and Palo Alto Networks are actively integrating GNNs to discover malicious infrastructure and enhance security observability, validating their real-world applicability [5, 4]. The core strength of GNNs lies in their ability to model information propagation across a network, making them exceptionally suited for detecting coordinated and multi-step attacks [19], a key feature of our system's correlation engine.

## 5.3 Explainability and Adversarial Robustness in GNNs

A significant challenge in applying complex models like GNNs to security is trust and interpretability. The field of explainable AI (XAI) for GNNs is an active area of research [26, 27]. Being able to explain \*why\* a GNN flagged a particular set of activities as malicious is crucial for a security analyst. Furthermore, like all machine learning models, GNNs are susceptible to adversarial attacks, where an attacker subtly modifies their behavior to evade detection [28, 29]. Research into defending GNNs and understanding their privacy implications is ongoing [30, 31]. While our current system's rule-based components are highly interpretable, a future GNN-based version would need to incorporate these principles of explainability and robustness.

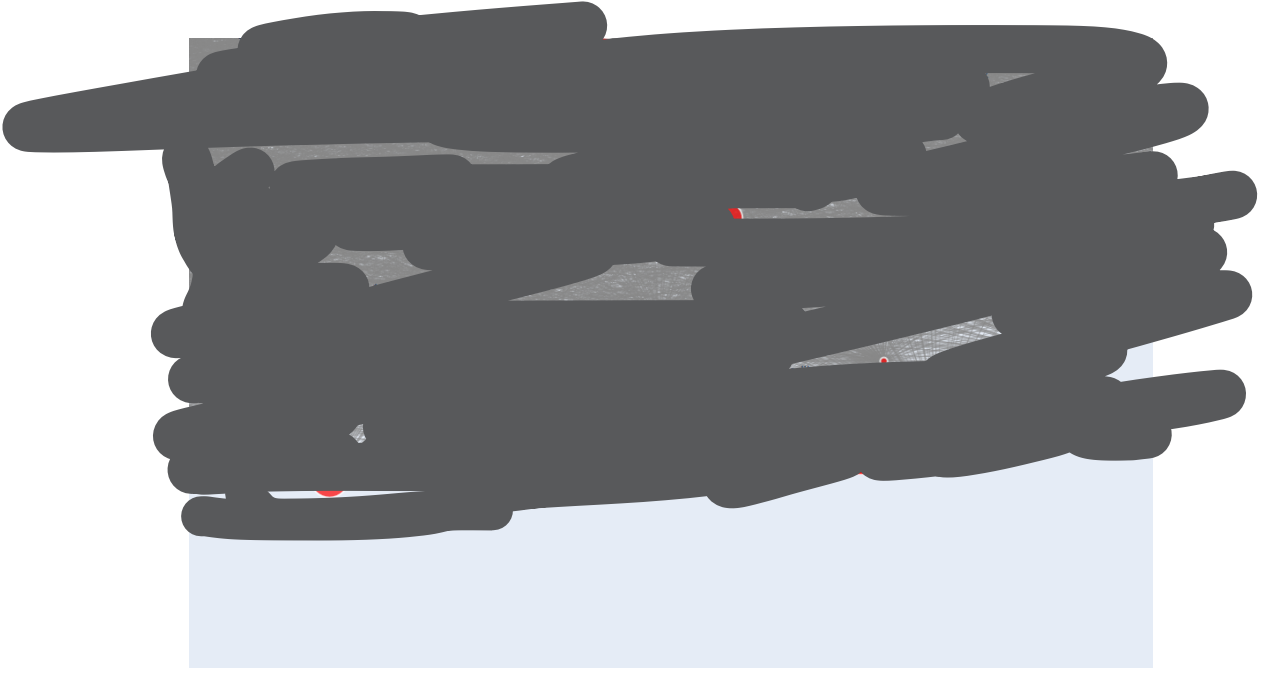


Figure 6: TH\_0004 Olası "Koordineli Saldırı" tehdidinin ağ grafiği. Grafikte, saldırıdan etkilenen 158 düğüm ve aralarındaki bağlantılar gösterilmektedir. Merkezi düğümler, saldırının başlangıç noktası veya en çok etkileşim alan sistemler olarak öne çıkmaktadır.

## References (Kaynakça)

### References

- [1] PuppyGraph. "Cyber Graph: Enhancing Cybersecurity with Graph Intelligence." *PuppyGraph Blog*, 2025. <https://www.puppygraph.com/blog/cyber-graph>
- [2] PuppyGraph. "SIEM Graph: Visualizing Security Data for Faster Threat Detection." *PuppyGraph Blog*, 2025. <https://www.puppygraph.com/blog/siem-graph>
- [3] Anonymous. "Graph Neural Networks in Anomaly Detection." *Chapter 26*, 2025. <https://graph-neural-networks.github.io/static/file/chapter26.pdf>
- [4] Palo Alto Networks Unit 42. "One Step Ahead in Cyber Hide-and-Seek: Automating Malicious Infrastructure Discovery With Graph Neural Networks." *Palo Alto Networks Blog*, 2025. <https://unit42.paloaltonetworks.com/graph-neural-networks/>
- [5] Splunk. "Leveraging Graph Neural Networks for Enhanced Security and Observability with Splunk." *Splunk Blog*, 2025. [https://www.splunk.com/en\\_us/blog/artificial-intelligence/splunk-graph-neural-networks-security-observability.html](https://www.splunk.com/en_us/blog/artificial-intelligence/splunk-graph-neural-networks-security-observability.html)
- [6] TigerGraph. "Real-Time Cyber Threat Detection with Graph DB." *TigerGraph Solutions*, 2025. <https://www.tigergraph.com/solutions-old/cybersecurity-threat-detection/>
- [7] Anonymous. "INTRUSION DETECTION USING GNN." *International Research Journal of Education and Technology*, 2025. <https://www.irjweb.com/INTRUSION%20DETECTION%20USING%20GNN.pdf>
- [8] Memgraph. "Using Graph Algorithms to Enhance Machine Learning for Cyber Threat Detection." *Memgraph Blog*, 2025. <https://memgraph.com/blog/using-graph-algorithms-to-enhance-machine-learning-cybersecurity>
- [9] Milvus. "What is graph-based anomaly detection?" *Milvus AI Quick Reference*, 2025. <https://milvus.io/ai-quick-reference/what-is-graphbased-anomaly-detection>
- [10] Algomox. "Graph Analytics for Advanced Threat Hunting." *Algomox Blog*, 2025. [https://www.algomox.com/resources/blog/graph\\_analytics\\_for\\_advanced\\_threat\\_hunting/](https://www.algomox.com/resources/blog/graph_analytics_for_advanced_threat_hunting/)
- [11] A. Al-Haija et al. "A Survey on Graph Neural Networks for Intrusion Detection Systems: Methods, Trends and Challenges." *ResearchGate*, 2024. [https://www.researchgate.net/publication/379292741\\_A\\_Survey\\_on\\_Graph\\_Neural\\_Networks\\_for\\_Intrusion\\_Detection\\_Systems\\_Methods\\_Trends\\_and\\_Challenges](https://www.researchgate.net/publication/379292741_A_Survey_on_Graph_Neural_Networks_for_Intrusion_Detection_Systems_Methods_Trends_and_Challenges)
- [12] S. He et al. "LogGD: Detecting Anomalies from System Logs by Graph Neural Networks." *arXiv:2209.07869*, 2022. <https://arxiv.org/abs/2209.07869>

- [13] Y. Wang et al. "Graph Neural Networks based Log Anomaly Detection and Explanation." *arXiv:2307.00527*, 2023. <https://arxiv.org/html/2307.00527v3>
- [14] H. Q. A. Al-Haija et al. "Systematic Review of Graph Neural Network for Malicious Attack Detection." *ResearchGate*, 2024. [https://www.researchgate.net/publication/392347657\\_Systematic\\_Review\\_of\\_Graph\\_Neural\\_Network\\_for\\_Malicious\\_Attack\\_Detection](https://www.researchgate.net/publication/392347657_Systematic_Review_of_Graph_Neural_Network_for_Malicious_Attack_Detection)
- [15] Y. Cai et al. "LogGC: Novel Approach for Graph-based Log Anomaly Detection." *ResearchGate*, 2024. [https://www.researchgate.net/publication/378025477\\_LogGC\\_Novel\\_Approach\\_for\\_Graph-based\\_Log\\_Anomaly\\_Detection](https://www.researchgate.net/publication/378025477_LogGC_Novel_Approach_for_Graph-based_Log_Anomaly_Detection)
- [16] Anonymous. "GNN-based Anomaly Detection for Encoded Network Traffic." *arXiv:2405.13670v1*, 2024. <https://arxiv.org/html/2405.13670v1>
- [17] CyberPoint International. "Graphing the Future: Harnessing the Power of Graph Neural Networks for Cybersecurity." *CyberPoint Blog*, 2025. <https://www.cyberpointllc.com/blog-posts/Graphing-the-Future-Harnessing-the-Power-of-Graph-Neural-Networks-for.php>
- [18] Anonymous. "(PDF) Cyber Security Attack Recognition on Cloud Computing Networks Based on Graph Convolutional Neural Network and GraphSage models." *ResearchGate*, 2024. [https://www.researchgate.net/publication/379990020\\_Cyber\\_Security\\_Attack\\_Recognition\\_on\\_Cloud\\_Computing\\_Networks\\_Based\\_on\\_Graph\\_Convolutional\\_Neural\\_Network\\_and\\_GraphSage\\_models](https://www.researchgate.net/publication/379990020_Cyber_Security_Attack_Recognition_on_Cloud_Computing_Networks_Based_on_Graph_Convolutional_Neural_Network_and_GraphSage_models)
- [19] F. Niu et al. "Detecting Complex Multi-step Attacks with Explainable Graph Neural Network." *arXiv:2405.11335v2*, 2024. <https://arxiv.org/html/2405.11335v2>
- [20] Anonymous. "FN-GNN: A Novel Graph Embedding Approach for Enhancing Graph Neural Networks in Network Intrusion Detection Systems." *MDPI Applied Sciences*, 2024. <https://www.mdpi.com/2076-3417/14/16/6932>
- [21] Anonymous. "CAGN-GAT Fusion: A Hybrid Contrastive Attentive Graph Neural Network for Network Intrusion Detection." *ResearchGate*, 2024. [https://www.researchgate.net/publication/389548085\\_CAGN-GAT\\_Fusion\\_A\\_Hybrid\\_Contrastive\\_Attentive\\_Graph\\_Neural\\_Network\\_for\\_Network\\_Intrusion\\_Detection](https://www.researchgate.net/publication/389548085_CAGN-GAT_Fusion_A_Hybrid_Contrastive_Attentive_Graph_Neural_Network_for_Network_Intrusion_Detection)
- [22] Anonymous. "Graph Anomaly Detection With Graph Neural Networks: Current Status and Challenges." *ResearchGate*, 2022. [https://www.researchgate.net/publication/364146164\\_Graph\\_Anomaly\\_Detection\\_with\\_Graph\\_Neural\\_Networks\\_Current\\_Status\\_and\\_Challenges](https://www.researchgate.net/publication/364146164_Graph_Anomaly_Detection_with_Graph_Neural_Networks_Current_Status_and_Challenges)
- [23] Anonymous. "Anomaly Detection in Cybersecurity with Graph-Based Approaches." *ResearchGate*, 2024. [https://www.researchgate.net/publication/383110497\\_Anomaly\\_Detection\\_in\\_Cybersecurity\\_with\\_Graph-Based\\_Approaches](https://www.researchgate.net/publication/383110497_Anomaly_Detection_in_Cybersecurity_with_Graph-Based_Approaches)

- [24] Anonymous. "Graph Neural Network Solutions for Anomaly Detection in Time Series." *ScholarSpace - University of Hawaii at Manoa*, 2023. <https://scholarspace.manoa.hawaii.edu/items/2154e7ea-9f12-4395-a90b-09e250fa8b9e>
- [25] USC Viterbi School of Engineering. "Spot the Difference: Safeguarding Cybersecurity with Graph Anomaly Detection." *USC Viterbi News*, 2025. <https://viterbischool.usc.edu/news/2025/01/spot-the-difference-safeguarding-cybersecurity-with-graph-anomaly-det>
- [26] flyingdoog. "awesome-graph-explainability-papers: Papers about explainability of GNNs." *GitHub Repository*, 2024. <https://github.com/flyingdoog/awesome-graph-explainability-papers>
- [27] R. Agarwal et al. "Explainable Graph Neural Networks Under Fire." *arXiv:2406.06417v2*, 2024. <https://arxiv.org/html/2406.06417v2>
- [28] V. L. T. Hoang et al. "Problem space structural adversarial attacks for Network Intrusion Detection Systems based on Graph Neural Networks." *arXiv:2403.11830*, 2024. <https://arxiv.org/abs/2403.11830>
- [29] Z. Zhang et al. "GNNGUARD: Defending Graph Neural Networks against Adversarial Attacks." *NeurIPS 2020*, 2020. <https://zitniklab.hms.harvard.edu/publications/papers/gnnguard-neurips20.pdf>
- [30] I. Ennadir et al. "Conformalized Adversarial Attack Detection for Graph Neural Networks." *Proceedings of Machine Learning Research*, 2023. <https://proceedings.mlr.press/v204/ennadir23a/ennadir23a.pdf>
- [31] J. Wu et al. "A Survey on Privacy in Graph Neural Networks: Attacks, Preservation, and Applications." *IEEE Transactions on Knowledge and Data Engineering*, 2024. <https://www.computer.org/csdl/journal/tk/2024/12/10693287/20vjvH76kU0>
- [32] Anonymous. "A Survey on Privacy Attacks and Defenses in Graph Neural Networks." *Information Technology and Control*, 2023. <https://itc.ktu.lt/index.php/ITC/article/view/37737/16750>
- [33] Anonymous. "(PDF) An Explainer for Temporal Graph Neural Networks." *ResearchGate*, 2023. [https://www.researchgate.net/publication/367047541\\_An\\_Explainer\\_for\\_Temporal\\_Graph\\_Neural\\_Networks](https://www.researchgate.net/publication/367047541_An_Explainer_for_Temporal_Graph_Neural_Networks)